

```

---
toc: false
---

<link rel="stylesheet" href="./styles/App.css">

```js
import {OVERRIDE_YEAR, RIVER_MILE, COOL_MIX_EVENTS} from "./config.js";

// Palette entries are paired with years oldest?newest (index 0 = 4 years
// back, last index = currentYear). All years now render as solid lines ?
// colors alone distinguish them. currentYear itself is computed below,
// once the data has loaded (see the "currentYear" cell) ? it no longer
// needs to be set by hand here.
const yearPalette = [
 {color: "#eda100", dash: []}, // oldest highlighted year
 {color: "#9085e9", dash: []},
 {color: "#3987e5", dash: []},
 {color: "#1baf7a", dash: []},
 {color: "#e34948", dash: []}, // currentYear
];

```js
// ?? Unit conversion ?????????????????????????????????????????????
// tmp is stored in °F throughout (see the parse step below) since that's
// the unit the chart, threshold comparisons, and historical stats are
// built around. toC() converts an already-Fahrenheit value back to °C
// purely for display, so it can be dropped in next to any °F reading
// without touching how anything is computed.
const toF = c => c * 9 / 5 + 32;
const toC = f => (f - 32) * 5 / 9;

```js
const raw = await FileAttachment("./data/RM15_water_temp.csv").text();
const parsed = d3.csvParse(raw, d => ({
 date: d3.timeParse("%Y-%m-%d")(d.date),
 tmp: d.tmp === "" ? null : toF(+d.tmp),
 cfs: d.cfs === "" ? null : +d.cfs
}));

```js
// ?? Season detection ?????????????????????????????????????????????
// currentYear is auto-detected as the newest year present in the data
// (falling back to OVERRIDE_YEAR from config.js if it's set). This is
// the ONLY thing that needs to change to move the whole page to a new
// season, and in normal operation it changes itself.
const yearsInData = Array.from(new Set(
  parsed.filter(d => d.date !== null).map(d => d.date.getFullYear())
));
const currentYear = OVERRIDE_YEAR ?? d3.max(yearsInData);

```js
const annotated = parsed
 .filter(d => d.tmp !== null && d.date !== null)
 .map(d => ({
 ...d,
 year: d.date.getFullYear(),
 doy: d3.utcDay.count(d3.utcYear(d.date), d.date),
 }));

```js
// yearPalette[i] is paired with (currentYear - (palette.length - 1 - i)),
// so the last palette entry always lands on currentYear.

```

```

const yearColors = Object.fromEntries(
  yearPalette.map((cfg, i) => [currentYear - (yearPalette.length - 1 - i), cfg])
);
```

```js
const historicalData = (() => {
  const byDoy = d3.group(annotated, d => d.doy);
  return Array.from(byDoy, ([doy, vals]) => {
    const temps = vals.map(v => v.tmp).sort(d3.ascending);
    return {
      doym: +doy,
      median: d3.quantile(temps, 0.50),
      p10: d3.quantile(temps, 0.10),
      p90: d3.quantile(temps, 0.90),
    };
  }).sort((a, b) => a.doy - b.doy);
})();
```

```js
const byYear = d3.group(annotated, d => d.year);
const highlightedYears = new Set(Object.keys(yearColors).map(Number));
const historicalYears = Array.from(byYear).filter(([year]) => !highlightedYears.has(year));
const highlightedYearRows = Array.from(byYear).filter(([year]) => highlightedYears.has(year));
```

```js
// ?? Cool Mix Flow windows ?????????????????????????????????????????????????????????????
// Converts each COOL_MIX_EVENTS on-ramp/off-ramp date (from config.js)
// into a day-of-year range, using the exact same "doy" convention as
// 'annotated' above (d3.utcDay.count(d3.utcYear(date), date)), so the
// windows compare directly against any highlighted year's row.doy.
//
// This is the ONLY thing needed to add or change a cool mix window ?
// everything downstream (the chart overlay, the legend) reads from
// this map and from COOL_MIX_EVENTS in config.js.
const coolMixWindowsByYear = new Map(
  COOL_MIX_EVENTS.map(ev => {
    const onDate = d3.timeParse("%Y-%m-%d")(ev.onRamp);
    const offDate = d3.timeParse("%Y-%m-%d")(ev.offRamp);
    return [ev.year, {
      onDoy: d3.utcDay.count(d3.utcYear(onDate), onDate),
      offDoy: d3.utcDay.count(d3.utcYear(offDate), offDate),
    }];
  })
);
```

```js
const dataCurrent = annotated.filter(d => d.year === currentYear).sort((a, b) => a.doy - b.doy);
const latest = dataCurrent[dataCurrent.length - 1];
const currentTemp = latest?.tmp;
const threshold = toF(15.5);
const daysAbove = dataCurrent.filter(d => d.tmp >= threshold).length;
const lastAbove = [...dataCurrent].reverse().find(d => d.tmp >= threshold);
const daysSinceAbove = lastAbove ? latest.doy - lastAbove.doy : null;

const monthNames = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
function doyToMonthDay(doy) {
  const d = new Date(currentYear, 0, 1 + doy);
  return `${monthNames[d.getMonth()]} ${d.getDate()}`;
}

```

```

const last7 = dataCurrent.slice(-7);
const prior7 = dataCurrent.slice(-14, -7);
const last7Avg = d3.mean(last7, d => d.tmp);
const prior7Avg = d3.mean(prior7, d => d.tmp);
const trendDelta = (prior7.length && last7.length) ? last7Avg - prior7Avg : null;
const trendDirection = trendDelta === null ? "flat" :
    trendDelta > 0.15 ? "rising" :
    trendDelta < -0.15 ? "falling" :
    "steady";
const trendArrow = trendDirection === "rising" ? "?" :
    trendDirection === "falling" ? "?" :
    "?";
const trendColor = trendDirection === "rising" ? "#B03823" :
    trendDirection === "falling" ? "#537F1C" :
    "#705C57";
const trendSub = trendDirection === "rising" ? "Warming vs. Prior Week" :
    trendDirection === "falling" ? "Cooling vs. Prior Week" :
    "Holding Steady";

const daysAboveSub = daysSinceAbove === 0 ? "Currently Above" :
    daysSinceAbove !== null ? `Last Exceeded ${daysSinceAbove} Day
s Ago` :
    "Not Yet Exceeded";

const status = currentTemp >= threshold ? "above" :
    currentTemp >= threshold - 1.8 ? "approaching" : // 1.8°F ? 1°C buffer
    "below";
const statusLabel = status === "above" ? "Threshold Exceeded" :
    status === "approaching" ? "Approaching Threshold" :
    "Below Threshold";
const lastUpdated = d3.max(parsed.filter(d => d.date !== null), d => d.date);
const lastUpdatedStr = latest?.date
    ? latest.date.toLocaleDateString("en-US", {year: "numeric", month: "long", day: "
numeric"})
    : "unknown";
```

```js
const logo = await FileAttachment("assets/LOGO.png").url();

display(htl.html`
<header class="site-header">
  
  <nav class="site-nav">
    <a href=".">Overview</a>
    <a href="./season">${currentYear} Season</a>
    <a href="./coolmix">Cool-Mix Flows</a>
  </nav>
</header>
`);

```js
display(htl.html`<div class="hero">
 <div class="hero-bg"></div>
 <div class="hero-content">
 <div class="hero-left">
 <p class="hero-eyebrow">Colorado River Mile ${RIVER_MILE} · Grand Canyon Nati
onal Park</p>
 <h1 class="hero-title">Daily Water Temperature</h1>
 <p class="hero-sub">Cool-Mix Threshold</p>
 </div>
 <div class="hero-right">
 <div class="hero-temp">${currentTemp?.toFixed(1)}<span class="hero-temp-unit"
>°F
 <div class="hero-temp-label">${statusLabel}</div>
 <div class="hero-temp-date">${latest?.date.toLocaleDateString("en-US", {month
: "short", day: "numeric"})}</div>

```

```

 </div>
 </div>
</div> `);
``

<div class="stat-row">
 <div class="stat-card stat-card--reading">
 <div class="stat-label">Last Reading Date</div>
 <div class="stat-value">${lastUpdatedStr}</div>
 <div class="stat-sub">Most Recent USGS Reading</div>
 </div>
 <div class="stat-card stat-card--trend">
 <div class="stat-label">7-day Trend</div>
 <div class="stat-value" style=${{color: trendColor}}>${trendArrow} ${trendDelta
 !== null ? Math.abs(trendDelta).toFixed(2) : "?"}°F</div>
 <div class="stat-sub">${trendSub}</div>
 </div>
 <div class="stat-card stat-card--days">
 <div class="stat-label">Days Above Threshold</div>
 <div class="stat-value">${daysAbove}</div>
 <div class="stat-sub">${daysAboveSub}</div>
 </div>
 <div class="stat-card stat-card--threshold">
 <div class="stat-label">Cool-Mix Threshold</div>
 <div class="stat-value">${threshold.toFixed(1)}°F ($
 {toC(threshold).toFixed(1)}°C)</div>
 <div class="stat-sub">Triggers Dam Release with Cool-Mix Flows</div>
 </div>
</div>

``js
const yearOptions = [...Object.keys(yearColors).sort((a, b) => b - a), "All"];

const focusYearEl = (() => {
 const root = htl.html`<div class="pill-group" role="radiogroup" aria-label="View"
 ></div>`;

 for (const opt of yearOptions) {
 // Note: swatch dots were removed from the pills themselves ? the
 // legend below the chart already maps each year to its color, so
 // repeating it here just added clutter and extra pill width.
 const btn = htl.html`<button
 type="button"
 class="pill"
 role="radio"
 aria-checked="false"
 >${opt}</button>`;
 btn.dataset.value = opt;
 root.appendChild(btn);
 }

 root.value = String(currentYear);
 root.querySelector(`[data-value="${currentYear}"]`).classList.add("pill-selected"
);
 root.querySelector(`[data-value="${currentYear}"]`).setAttribute("aria-checked",
"true");

 for (const btn of root.querySelectorAll(".pill")) {
 btn.onclick = () => {
 const alreadySelected = btn.classList.contains("pill-selected");

 for (const b of root.querySelectorAll(".pill")) {
 b.classList.remove("pill-selected");
 b.setAttribute("aria-checked", "false");
 }

 if (alreadySelected) {
 // Clicking the active pill again turns it off ? no year pill selected

```

```

 root.value = null;
 } else {
 btn.classList.add("pill-selected");
 btn.setAttribute("aria-checked", "true");
 root.value = btn.dataset.value;
 }

 root.dispatchEvent(new Event("input", {bubbles: true}));
};
}

return root;
})();

const focusYear = view(focusYearEl);
```

```js
function pillToggle(label, initial) {
 const btn = htl.html`<button type="button" class="toggle ${initial ? "toggle-on"
: ""}" role="switch" aria-checked="${initial}">

 ${label}
</button>`;

 const root = htl.html`<span class="toggle-wrap">${btn}</span>`;
 root.value = initial;

 btn.onclick = () => {
 root.value = !root.value;
 btn.classList.toggle("toggle-on", root.value);
 btn.setAttribute("aria-checked", String(root.value));
 root.dispatchEvent(new Event("input", {bubbles: true}));
 };

 return root;
}

const medianToggleEl = pillToggle("Historical Median", true);
const showMedian = view(medianToggleEl);
```

```js
const bandToggleEl = pillToggle("10th?90th Percentile band", true);
const showBand = view(bandToggleEl);
```

```js
// "Historical" moved out of the main pill-group and into the toggle row,
// styled to match the median/band toggle switches it's grouped with.
// It's now an independent overlay: turning it on/off doesn't touch and
// isn't touched by whichever year pill is currently selected.
const historicalToggleEl = pillToggle("Historical", false);
const showHistorical = view(historicalToggleEl);
```

```js
// Cool Mix Flows toggle ? independently controls whether the soft "halo"
// glow overlay (drawn per highlighted year from coolMixWindowsByYear /
// COOL_MIX_EVENTS) is shown on the chart. This is separate from which
// year is focused: turning it off hides the halo for whichever year(s)
// are currently focused (e.g. 2024, 2025) without touching the base
// temperature lines. It also drives whether the "Cool Mix Flows Active"
// legend entry appears, on-page and in exports.
const coolMixToggleEl = pillToggle("Cool-Mix Flows", true);
const showCoolMix = view(coolMixToggleEl);
```

```

```

```js
display(htl.html`<div class="controls-row">
 <div class="controls-left">${focusYearEl}</div>
 <div class="controls-right toggle-row">${historicalToggleEl}${medianToggleEl}${bandToggleEl}${coolMixToggleEl}</div>
</div>`);
```

```js
const chartPlot = resize((width) => Plot.plot({
 width,
 height: 500,
 marginLeft: 60,
 marginBottom: 45,
 style: {
 fontFamily: "IBM Plex Mono, monospace",
 fontSize: "16px",
 background: "transparent",
 },
 x: {
 tickFormat: doy => {
 const monthStarts = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334];
 const monthNames = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
 const i = monthStarts.indexOf(doy);
 return i >= 0 ? monthNames[i] : null;
 },
 ticks: [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334],
 label: null,
 },
 y: {
 label: "Water temperature (°F)",
 domain: [toF(7), toF(22)],
 },
 marks: [
 Plot.rectY([{}], {
 x1: 0, x2: 365, y1: threshold, y2: toF(22),
 fill: "#B03823", fillOpacity: 0.07,
 }),
 ...(showBand ? [Plot.areaY(historicalData, {
 x: "doy", y1: "p10", y2: "p90",
 fill: "#90A98B", fillOpacity: 0.22,
 })] : []),
 ...(showMedian ? [Plot.line(historicalData, {
 x: "doy", y: "median",
 stroke: "#57423E", strokeWidth: 1.5, strokeDasharray: "6 3",
 })] : []),
 Plot.ruleY([threshold], {
 stroke: "#B03823", strokeDasharray: "4 2", strokeWidth: 1.2,
 }),
 // Threshold value labeled neatly right above its dashed rule line.
 // °F leads (matches the axis and every other on-chart reading);
 // °C follows in parentheses.
 Plot.text([{}], {
 x: 364, y: threshold,
 text: () => `${threshold.toFixed(1)}°F`,
 dy: -8,
 textAnchor: "end",
 fontSize: 13,
 fontWeight: 500,
 fill: "#B03823",
 }),
 ...(showHistorical ? historicalYears.map(([year, rows]) =>
 Plot.line(rows, {
 x: "doy", y: "tmp",
 stroke: "#705C57", strokeWidth: 0.8, strokeOpacity: 0.35,
 class: `hist-line hist-year-${year}`,
 })
) : [])
]
});

```

```

) : []),
...(showHistorical ? [Plot.tip(
 historicalYears.flatMap(([year, rows]) => rows.map(d => ({...d, year}))),
 Plot.pointer({
 x: "doy", y: "tmp",
 title: d => `${d.year} ? ${doyToMonthDay(d.doy)}\n${d.tmp.toFixed(1)}°F`,
 })
)] : []),
// Cool Mix Flow halo: a soft, wide, low-opacity glow drawn UNDERNEATH
// the base temperature line, but only across each highlighted year's
// on-ramp?off-ramp window (as defined by COOL_MIX_EVENTS in
// config.js). Because this halo is drawn before (i.e. beneath) the
// base line mark directly below, the crisp colored line still reads
// on top, with a soft glow "around" it only during the cool mix
// window ? instead of the old heavier dashed line drawn over the top.
// It reuses that year's own color and mirrors the same focus/
// non-focus opacity split as the base line, so switching the focused
// year or toggling "All" doesn't fight with this overlay ? it just
// rides along with it. Gated on showCoolMix so the "Cool Mix Flows"
// toggle can turn this styling on/off independently (e.g. for 2024 /
// 2025) without affecting the base temperature lines.
...(focusYear && showCoolMix ? highlightedYearRows.flatMap(([year, rows]) => {
 const window = coolMixWindowsByYear.get(year);
 if (!window) return [];
 const coolRows = rows.filter(d => d.doy >= window.onDoy && d.doy <= window.of
fDoy);
 if (coolRows.length < 2) return [];
 const isFocus = focusYear === "All" || String(year) === focusYear;
 // Two stacked strokes build the halo: a wide, very soft outer glow
 // plus a slightly tighter, slightly stronger inner glow, so the
 // falloff feels soft rather than a single flat band of opacity.
 return [
 Plot.line(coolRows, {
 x: "doy", y: "tmp",
 stroke: yearColors[year].color,
 strokeWidth: isFocus ? 16 : 8,
 strokeOpacity: isFocus ? 0.12 : 0.05,
 strokeLinecap: "round",
 strokeLinejoin: "round",
 class: `coolmix-halo coolmix-halo-outer coolmix-year-${year}`,
 }),
 Plot.line(coolRows, {
 x: "doy", y: "tmp",
 stroke: yearColors[year].color,
 strokeWidth: isFocus ? 9 : 4.5,
 strokeOpacity: isFocus ? 0.28 : 0.1,
 strokeLinecap: "round",
 strokeLinejoin: "round",
 class: `coolmix-halo coolmix-halo-inner coolmix-year-${year}`,
 }),
];
}) : []),
...(focusYear ? highlightedYearRows.map(([year, rows]) => {
 const isFocus = focusYear === "All" || String(year) === focusYear;
 return Plot.line(rows, {
 x: "doy", y: "tmp",
 stroke: yearColors[year].color,
 strokeWidth: isFocus ? 3 : 1.0,
 strokeOpacity: isFocus ? 1 : 0.2,
 strokeDasharray: (yearColors[year]?.dash ?? []).join(" "),
 });
}) : []),
...(focusYear ? [Plot.text(
 () => {
 const peaks = highlightedYearRows
 .filter(([year]) => focusYear === "All" || String(year) === focusYear)
 .map(([year, rows]) => {
 const peak = rows.reduce((a, b) => b.tmp > a.tmp ? b : a);

```

```

 return { doy: peak.doy, tmp: peak.tmp, label: String(year) };
 })
 .sort((a, b) => a.doy - b.doy);

 // Nudge labels apart vertically when they're close in both x and y
 const minGap = 1.3;
 const xWindow = 25;
 for (let i = 1; i < peaks.length; i++) {
 for (let j = 0; j < i; j++) {
 if (Math.abs(peaks[i].doy - peaks[j].doy) < xWindow &&
 Math.abs(peaks[i].tmp - peaks[j].tmp) < minGap) {
 peaks[i].tmp = peaks[j].tmp + minGap;
 }
 }
 }

 return peaks.map(p => ({...p, tmp: p.tmp + 0.7}));
 })(),
 {
 x: "doy", y: "tmp", text: "label", fontSize: 16, fontWeight: 300,
 fill: d => yearColors[d.label]?.color ?? "#2C0E09",
 stroke: "#FEECD8", strokeWidth: 3, paintOrder: "stroke",
 }
]) : []),
 ...(focusYear === "All" || focusYear === String(currentYear) ? [Plot.dot(dataCu
rrent.slice(-1), {
 x: "doy", y: "tmp",
 r: 5, fill: "#2C0E09", stroke: "#ffffff", strokeWidth: 1, /* Current Temperature
Dot*/
 })] : []),
 ...(focusYear ? (() => {
 const tipYear = focusYear === "All" ? currentYear : focusYear;
 const tipRows = highlightedYearRows.find(([year]) => String(year) === String(
tipYear))?.[1];
 return tipRows ? [Plot.tip(tipRows, Plot.pointerX({
 x: "doy", y: "tmp",
 title: d => `${tipYear} ? ${doyToMonthDay(d.doy)}\n${d.tmp.toFixed(1)}°F`,
 }))] : [];
 })() : []),
],
));
display(html.html`<div class="chart-card">${chartPlot}</div>`);
```

```js
// Hover-to-highlight for historical lines. Listens on the resize wrapper
// (which persists across width changes) and re-queries the current <svg>
// each time, so it keeps working even after a resize swaps the inner markup.
if (showHistorical) {
 const histRows = historicalYears.flatMap(([year, rows]) => rows.map(d => ({...d,
year})));

 chartPlot.addEventListener("pointermove", (event) => {
 const svgEl = chartPlot.querySelector("svg");
 const xScale = svgEl?.scale?.("x");
 if (!svgEl || !xScale) return;

 const [px] = d3.pointer(event, svgEl);
 const doy = xScale.invert(px);

 let nearest = null, nearestDist = Infinity;
 for (const d of histRows) {
 const dist = Math.abs(d.doy - doy);
 if (dist < nearestDist) { nearestDist = dist; nearest = d; }
 }

 svgEl.querySelectorAll(".hist-line").forEach(el => el.classList.remove("hist-li
ne-active"));
 });
}

```

```

 if (nearest && nearestDist < 6) {
 svgEl.querySelectorAll(`.hist-year-${nearest.year}`).forEach(el => el.classList.add("hist-line-active"));
 }
 });

 chartPlot.addEventListener("pointerleave", () => {
 chartPlot.querySelectorAll(".hist-line").forEach(el => el.classList.remove("hist-line-active"));
 });
}
```

```js
const legendEl = htl.html`<div class="legend">
 <div class="legend-items">
 ${Object.entries(yearColors).sort((a, b) => b[0] - a[0]).map(([year, cfg]) => {
 const dashAttr = cfg.dash.length ? cfg.dash.join(",") : "none";
 return htl.html`<div class="legend-item">
 <svg width="32" height="12">
 <line x1="0" y1="6" x2="32" y2="6"
 stroke="${cfg.color}"
 stroke-width="2.5"
 stroke-dasharray="${dashAttr}" />
 </svg>
 ${year}
 </div>`;
 })}
 ${showCoolMix ? htl.html`<div class="legend-item legend-item--coolmix">
 <svg width="32" height="12">
 <line x1="0" y1="6" x2="32" y2="6" stroke="#2C0E09" stroke-width="9" stroke-opacity="0.2" stroke-linecap="round"/>
 <line x1="0" y1="6" x2="32" y2="6" stroke="#2C0E09" stroke-width="2.5" stroke-linecap="round"/>
 </svg>
 Cool-Mix Flows Active
 </div>` : null}
 <div class="legend-item">
 <svg width="32" height="12">
 <line x1="0" y1="6" x2="32" y2="6" stroke="#705C57" stroke-width="2.5"/>
 </svg>
 Historical
 </div>
 <div class="legend-item">
 <svg width="32" height="12">
 <line x1="0" y1="6" x2="32" y2="6" stroke="#57423E" stroke-width="1.5" stroke-dasharray="6,3"/>
 </svg>
 Median
 </div>
 <div class="legend-item">
 <svg width="32" height="12">
 <rect x="0" y="2" width="32" height="8" fill="#93A87B" opacity="0.4" rx="2" />
 </svg>
 10th?90th Percentile
 </div>
 <div class="legend-item">
 <svg width="32" height="12">
 <line x1="0" y1="6" x2="32" y2="6" stroke="#B03823" stroke-width="1.5" stroke-dasharray="4,2"/>
 </svg>
 ${threshold.toFixed(1)}°F Threshold
 </div>
 </div>`;
display(legendEl);
```

```

```

``js
// ?? SVG / JPEG export ?????????????????????????????????????????
// Builds a standalone SVG string: logo + hero + stat cards header, then
// the chart, then a legend laid out with a measured-width flow (so labels
// of different lengths never overlap), then the usage credit line.
//
// This is shared by both export buttons below: the SVG button downloads
// the string directly, and the JPEG button rasterizes the same string
// onto a canvas before downloading it as a JPEG.

function escapeXml(str) {
    return String(str).replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
}

function estimateTextWidth(text, fontSize) {
    return text.length * fontSize * 0.56;
}

function wrapTextToLines(text, maxWidth, fontSize) {
    const words = text.split(" ");
    const lines = [];
    let current = "";
    for (const word of words) {
        const test = current ? `${current} ${word}` : word;
        if (current && estimateTextWidth(test, fontSize) > maxWidth) {
            lines.push(current);
            current = word;
        } else {
            current = test;
        }
    }
    if (current) lines.push(current);
    return lines;
}

async function loadLogoForExport() {
    const resp = await fetch(logo);
    const blob = await resp.blob();
    const dataUri = await new Promise((resolve, reject) => {
        const reader = new FileReader();
        reader.onload = () => resolve(reader.result);
        reader.onerror = reject;
        reader.readAsDataURL(blob);
    });
    const dims = await new Promise((resolve, reject) => {
        const img = new Image();
        img.onload = () => resolve({ width: img.naturalWidth, height: img.naturalHeight });
        img.onerror = reject;
        img.src = dataUri;
    });
    return { dataUri, ...dims };
}

// Builds the full export SVG and returns both the markup string and its
// pixel dimensions (the JPEG exporter needs the dimensions to size its
// canvas correctly).
async function buildExportSvg() {
    const chartSvg = chartPlot.querySelector("svg");
    if (!chartSvg) return null;

    const chartClone = chartSvg.cloneNode(true);
    const chartWidth = chartClone.viewBox.baseVal.width || chartClone.getBoundingBoxClientRect().width;
    const chartHeight = chartClone.viewBox.baseVal.height || chartClone.getBoundingBoxClientRect().height;

```

```

const padX = 24;

// ?? Logo ?????????????????????????????????????????????????????????????
const logoInfo = await loadLogoForExport();
const logoHeight = 50;
const logoWidth = logoInfo.width && logoInfo.height ? (logoInfo.width / logoInfo.
height) * logoHeight : logoHeight;

// ?? Header: logo, title, current reading ?????????????????????
const titleLine1 = "Daily Water Temperature";
const titleLine2 = "Smallmouth Bass Spawning Threshold";
const titleX = padX + logoWidth + 24;
const heroRightX = chartWidth - padX;
const currentTempStr = currentTemp !== undefined ? `${currentTemp.toFixed(1)}°F`
: "?";
const currentTempCStr = currentTemp !== undefined ? `${toC(currentTemp).toFixed(1
)}°C` : "";
const heroDateStr = latest?.date ? latest.date.toLocaleDateString("en-US", { mont
h: "short", day: "numeric" }) : "";

const headerTopSvg = `
  <image href="${logoInfo.dataUri}" x="${padX}" y="24" width="${logoWidth}" heigh
t="${logoHeight}" preserveAspectRatio="xMinYMin meet"/>
  <text x="${titleX}" y="40" font-family="Source Sans 3, sans-serif" font-size="1
3" letter-spacing="2" fill="#705C57">${escapeXml('COLORADO RIVER MILE ${RIVER_MILE}
  • GRAND CANYON NATIONAL PARK')}</text>
  <text x="${titleX}" y="66" font-family="PT Serif, Georgia, serif" font-size="24
" font-weight="600" fill="#251F21">${escapeXml(titleLine1)}</text>
  <text x="${titleX}" y="86" font-family="PT Serif, Georgia, serif" font-size="14
" font-style="italic" fill="#705C57">${escapeXml(titleLine2)}</text>
  <text x="${heroRightX}" y="50" text-anchor="end" font-family="IBM Plex Mono, mo
nospace" font-weight="500" fill="#B03823"><tspan font-size="32">${escapeXml(current
TempStr)}</tspan><tspan font-size="18" opacity="0.5" dx="6"></tspan><tspan font-si
ze="18" opacity="0.75" dx="6">${escapeXml(currentTempCStr)}</tspan></text>
  <text x="${heroRightX}" y="70" text-anchor="end" font-family="Source Sans 3, sa
ns-serif" font-size="14" letter-spacing="1" fill="#B03823">${escapeXml(statusLabel.
toUpperCase())}</text>
  <text x="${heroRightX}" y="88" text-anchor="end" font-family="Source Sans 3, sa
ns-serif" font-size="13" fill="#251F21">${escapeXml(heroDateStr)}</text>
`;

const ruleY = 110;

// ?? Stat row ?????????????????????????????????????????????????????????
const statEntries = [
  { label: "LAST READING DATE", value: lastUpdatedStr, sub: "Most Recent USGS Rea
ding" },
  { label: "7-DAY TREND", value: `${trendArrow} ${trendDelta !== null ? Math.abs(
trendDelta).toFixed(2) : "?"}°F`, valueColor: trendColor, sub: trendSub },
  { label: "DAYS ABOVE THRESHOLD", value: String(daysAbove), sub: daysAboveSub },
  { label: "COOL-MIX THRESHOLD", value: `${threshold.toFixed(1)}°F (${toC(thresho
ld).toFixed(1)}°C)`, sub: "Triggers Dam Release with Cool-Mix Flows" },
];
const statTop = ruleY + 24;
const colWidth = (chartWidth - padX * 2) / statEntries.length;
const statSvg = statEntries.map((s, i) => {
  const x = padX + i * colWidth;
  return `
    <text x="${x}" y="${statTop + 14}" font-family="PT Serif, Georgia, serif" fon
t-size="11" letter-spacing="1.5" fill="#251F21">${escapeXml(s.label)}</text>
    <text x="${x}" y="${statTop + 38}" font-family="IBM Plex Mono, monospace" fon
t-size="20" font-weight="500" fill="${s.valueColor ?? "#2C0E09"}">${escapeXml(s.val
ue)}</text>
    <text x="${x}" y="${statTop + 56}" font-family="Source Sans 3, sans-serif" fo
nt-size="12" fill="#000000">${escapeXml(s.sub)}</text>
  `;
}).join("");
const statBottom = statTop + 66;

```



```

    const creditText = "Graphs may be used for non-commercial purposes provided that
they are not altered or edited and they are appropriately credited to the Grand Can
yon Trust.";
    const creditLines = wrapTextToLines(creditText, chartWidth - padX * 2, 12);
    const creditSvg = creditLines.map((line, i) =>
    '<text x="${padX}" y="${20 + i * 16}" font-family="PT Serif, Georgia, serif" fo
nt-style="italic" font-size="12" fill="#AC9E9B">${escapeXml(line)}</text>'
    ).join("");
    const creditHeight = creditLines.length * 16 + 16;

    const totalHeight = headerHeight + chartHeight + legendHeight + creditHeight;

    // JPEG has no transparency, so the export SVG always carries an explicit
    // white background rect (it already does, below) ? no change needed for
    // that reason, but it's what lets the canvas rasterization look right.
    const combined = '<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www
.w3.org/1999/xlink" width="${chartWidth}" height="${totalHeight}" viewBox="0 0 ${ch
artWidth} ${totalHeight}">
    <rect width="${chartWidth}" height="${totalHeight}" fill="#ffffff"/>
    <g>${headerTopSvg}</g>
    <line x1="${padX}" y1="${ruleY}" x2="${chartWidth - padX}" y2="${ruleY}" stroke
="#E1B9A6" stroke-width="1"/>
    <g>${statSvg}</g>
    <g transform="translate(0, ${headerHeight})">${chartClone.innerHTML}</g>
    <g transform="translate(8, ${headerHeight + chartHeight + 8})">${legendItemsSvg
}</g>
    <g transform="translate(0, ${headerHeight + chartHeight + legendHeight})">${cre
ditSvg}</g>
    </svg>`;

    return { svgString: combined, width: chartWidth, height: totalHeight };
}

function exportFilenameBase() {
    const todayStr = new Date().toISOString().slice(0, 10);
    return `mile-${RIVER_MILE}-temp-${(focusYear ?? "none").toLowerCase()}-${todayStr
}`;
}

// ?? SVG download button ?????????????????????????????????????????????????????
const exportSvgBtn = htl.html`<button class="export-btn">? Download chart as SVG</button>`;

exportSvgBtn.onclick = async () => {
    const originalLabel = exportSvgBtn.textContent;
    exportSvgBtn.textContent = "Preparing?";
    exportSvgBtn.disabled = true;

    try {
        const built = await buildExportSvg();
        if (!built) return;

        const blob = new Blob([built.svgString], { type: "image/svg+xml" });
        const a = document.createElement("a");
        a.href = URL.createObjectURL(blob);
        a.download = `${exportFilenameBase()}.svg`;
        a.click();
        URL.revokeObjectURL(a.href);
    } finally {
        exportSvgBtn.textContent = originalLabel;
        exportSvgBtn.disabled = false;
    }
};

// ?? JPEG download button ?????????????????????????????????????????????????????
// Reuses the exact same SVG the SVG button downloads, then rasterizes
// it onto an off-screen canvas (at 2x scale for crisper output on
// retina displays / printing) and exports that as a JPEG.
const exportJpegBtn = htl.html`<button class="export-btn">? Download chart as JPEG</button>`;

```

```

exportJpegBtn.onclick = async () => {
  const originalLabel = exportJpegBtn.textContent;
  exportJpegBtn.textContent = "Preparing?";
  exportJpegBtn.disabled = true;

  let svgUrl;
  try {
    const built = await buildExportSvg();
    if (!built) return;

    const scale = 2; // render at 2x so the JPEG stays crisp at full size
    const svgBlob = new Blob([built.svgString], { type: "image/svg+xml;charset=utf-
8" });
    svgUrl = URL.createObjectURL(svgBlob);

    const img = new Image();
    await new Promise((resolve, reject) => {
      img.onload = resolve;
      img.onerror = reject;
      img.src = svgUrl;
    });

    const canvas = document.createElement("canvas");
    canvas.width = Math.ceil(built.width * scale);
    canvas.height = Math.ceil(built.height * scale);
    const ctx = canvas.getContext("2d");

    // JPEG has no transparency channel, so flatten onto a white
    // background before drawing the chart on top of it.
    ctx.fillStyle = "#ffffff";
    ctx.fillRect(0, 0, canvas.width, canvas.height);
    ctx.scale(scale, scale);
    ctx.drawImage(img, 0, 0, built.width, built.height);

    const jpegBlob = await new Promise(resolve => canvas.toBlob(resolve, "image/jpe
g", 0.92));
    if (!jpegBlob) throw new Error("Canvas failed to produce a JPEG blob");

    const a = document.createElement("a");
    a.href = URL.createObjectURL(jpegBlob);
    a.download = `${exportFilenameBase()}.jpg`;
    a.click();
    URL.revokeObjectURL(a.href);
  } finally {
    if (svgUrl) URL.revokeObjectURL(svgUrl);
    exportJpegBtn.textContent = originalLabel;
    exportJpegBtn.disabled = false;
  }
};

display(html`<div class="export-row">${exportSvgBtn}${exportJpegBtn}</div>`);
`);

```

```

<p class="last-updated">
Curious about water temperatures in the Grand Canyon? This tool tracks the daily wa
ter temperature of the Colorado River at river mile 15, a few miles downstream of S
oap Creek Rapid.
</p>

```

```

<p class="last-updated">
In recent years, river temperatures have been trending warmer as declining water le
vels in Lake Powell have caused warmer water to be released from Glen Canyon Dam. U
se the graph to see how river temperatures change over time and understand when the
y reach an important threshold.
</p>

```

```

<p class="last-updated">

```

At 59.5 ± (15.5 ±), conditions warrant cool-mix flows, a management tool introduced in 2024 t
</p>

```
`` `js
display(htl.html `
  <footer class="site-footer">
    <p class="site-footer-citation">
      Observations from
      <a href="https://waterdata.usgs.gov/monitoring-location/USGS-09380000"
        target="_blank"
        rel="noopener noreferrer">
        USGS Gage 09380000
      </a>
      • Temperature estimates based on
      <a href="https://doi.org/10.1002/eap.2279"
        target="_blank"
        rel="noopener noreferrer">
        Dibble et al. (2020)
      </a>
      thermal model.
    </p>
    <p class="site-footer-credit">
      Graphs may be used for non-commercial purposes provided that they are not alt
      ered or edited and are appropriately credited to the Grand Canyon Trust.
    </p>
  </footer>
`);
`` `
```